

Part 3: Event Generators

PYTHIA, HERWIG, and the Monte Carlo Bridge Between Theory and Data

PHYS 7502 – Final Reading Assignment

1 Why Do We Need Event Generators At All?

Here’s a question that might have been nagging at you since Part 2. Jet algorithms are defined as computer programs – they literally need an input list of 4-vectors to run on. If jets are the main way we connect QCD theory to detector data, how do we test them *before* running them on real data? How do we know anti- k_T behaves the way the theorists claim? How do we calibrate the detector’s jet energy response?

The answer is **simulated events**. We need a computer program that takes some physical process as input (say, $pp \rightarrow Z + \text{jets}$ at $\sqrt{s} = 13 \text{ TeV}$) and outputs realistic, detailed events – lists of 4-vectors and particle IDs, event by event, as close to what a real detector would see as possible.

What an Event Generator Is

An **event generator** is a computer program that simulates particle collisions from first principles (QCD, electroweak, etc.), producing on output a list of final-state particles with their 4-momenta, particle IDs, and sometimes additional information about the history of the event (which parton came from which, etc.).

Because physics is *probabilistic* at this level (it’s quantum mechanics after all), the program is fundamentally a **Monte Carlo**: it uses random-number sampling to pick each particle’s properties according to the appropriate probability distributions derived from theory.

Think Of It As A Physics Movie

An event generator is like a director producing individual “movies” of collisions. Each run produces a different event – different random kinematics, different number of particles, different species – but all drawn from the correct underlying distributions. Run the generator a million times and you have a statistical sample of what the LHC would produce for that process, which you can push through detector simulation and analyze just like real data.

2 The Four-Step Structure of an Event

Every modern event generator, regardless of vendor, follows roughly the same four-step recipe. This structure is *imposed* by the factorization theorems of QCD we talked about in Part 4 (fragmentation functions). Each step handles a different physical regime.

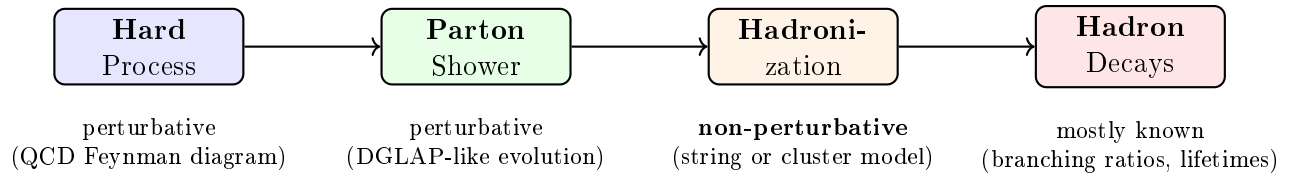


Figure 1: The four-step generation recipe. The first two steps are computed perturbatively from QCD; the third is the genuinely non-perturbative hadronization step, which requires a phenomenological model; the fourth is largely data-driven (use measured decay widths and branching fractions).

Let's walk through each step.

2.1 Step 1: the hard process

You pick the process you want to simulate – say, $gg \rightarrow H$, or $q\bar{q} \rightarrow Z$, or just generic inclusive pp scattering. For the chosen process, the generator:

1. Uses **parton distribution functions (PDFs)** to pick the momentum fractions x_1, x_2 of the incoming partons.
2. Uses the **perturbative QCD matrix element** to compute the cross section and pick outgoing parton kinematics.

For tree-level (leading order, LO) calculations, this is straightforward. For NLO or higher, it's more involved.

2.2 Step 2: the parton shower

The hard partons from Step 1 aren't the end of the story. They carry a large virtuality from the hard scattering, and they shed that virtuality by emitting additional (softer, more collinear) partons. This is the **parton shower**.

Physically: a quark of virtuality $\sim Q$ can split into a quark plus a gluon of virtuality $\sim Q/2$; each of those can split again; and so on, until you reach the hadronization scale $\sim 1 \text{ GeV}$ where everything is soft and collinear enough that perturbation theory breaks down.

The shower is a Markov process governed by **splitting functions** like the DGLAP splitting functions $P_{q \rightarrow qg}$, $P_{g \rightarrow q\bar{q}}$, $P_{g \rightarrow gg}$.

Shower as Inverse DGLAP

Remember the DGLAP equation for fragmentation functions from Part 4? It told you how $D_i^h(z, Q^2)$ evolves with Q^2 . A parton shower is essentially the *stochastic* version of DGLAP evolution, run at the level of individual events. Instead of evolving a distribution, you pick concrete splittings at random according to the probabilities built into the splitting functions. Run many events, and you reproduce the distribution.

2.3 Step 3: hadronization

At some low scale $Q_0 \sim 1 \text{ GeV}$, the shower stops and you have a bunch of colored partons. Now you need to turn them into hadrons. This is the **genuinely non-perturbative** part, and it's where different generators make different choices:

- PYTHIA uses the **Lund string model** (Part 6).

- The (retired) HERWIG uses the **cluster model**.

2.4 Step 4: hadron decays

Finally, many of the hadrons produced by hadronization are unstable. A ρ^0 decays to $\pi^+\pi^-$ in 10^{-23} s; a K_S^0 decays to $\pi^+\pi^-$ or $\pi^0\pi^0$ in 10^{-10} s; a π^0 decays to $\gamma\gamma$ in 10^{-16} s. These decays are handled by the generator using known branching ratios and matrix elements.

The final state after Step 4 is the list of *detector-stable* particles: $e^\pm$, $\mu^\pm$, γ , $\pi^\pm$, $K^\pm$, K_L^0 , p , $\bar{p}$, n , $\bar{n}$, and neutrinos.

3 PYTHIA: The Dominant Event Generator

PYTHIA has been the world’s dominant LHC event generator since about 2005 – five years before the LHC turned on. It was written primarily by Törbjörn Sjöstrand and collaborators at Lund University in Sweden (the same “Lund” that gives the Lund string model its name – this is not a coincidence).

PYTHIA at a Glance

- **Official web page:** <https://pythia.org/>
- **Order:** Leading order (LO) for matrix elements.
- **Shower:** p_T -ordered dipole shower.
- **Hadronization:** Lund string model.
- **Beams supported:** any combination of $e^\pm$, $\mu^\pm$, p , $\bar{p}$, n , $\dots$ (but *not* nuclei – that’s what HIJING is for).
- **Inputs needed:** PDFs (usually from an external library), process choice, center-of-mass energy.

3.1 What “leading order” means for PYTHIA

PYTHIA computes the hard-process matrix element at **tree level**. So for $q\bar{q} \rightarrow W^+$, it uses the tree-level amplitude (one Feynman diagram, no loops). This is fast and reliable for inclusive-type quantities but misses next-to-leading-order contributions that can be important (e.g. for precision rate predictions).

However, PYTHIA’s *parton shower* resums large logarithms arising from multiple collinear emissions. So while the matrix element is LO, the shower is effectively all-orders in those logarithms. For most jet physics, this is the dominant effect.

3.2 Tuning

The Lund string model has *parameters* – the string tension κ , the a and b parameters in the splitting function (see Part 6), parameters controlling p_T widths, heavy-quark suppression factors, etc. These parameters aren’t determined from first principles; they’re **tuned** to data.

The Tuning Game

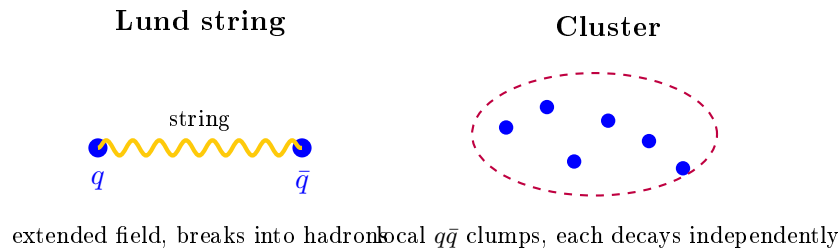
Every major LHC experiment (ATLAS, CMS, LHCb, ALICE) produces its own “PYTHIA tune” – a set of parameter values that give best agreement with that experiment’s data (charged particle spectra, multiplicities, underlying event properties, etc.). Tunes evolve as new data comes in and as PYTHIA itself improves. The most famous tune ancestry goes: $Z2 \rightarrow A14 \rightarrow Monash \rightarrow \dots$. If someone says “we used the Monash tune”, they mean a specific set of PYTHIA parameter values that were fit to a specific set of measurements.

4 HERWIG: The Cluster-Model Alternative (Retired)

HERWIG (short for “Hadron Emission Reactions With Interfering Gluons”) was for decades the main alternative to PYTHIA. Its main difference was using the **cluster model** for hadronization instead of the string model.

The cluster model:

1. After the parton shower, force all gluons to split non-perturbatively into $q\bar{q}$ pairs.
2. Group the resulting quarks into color-singlet $q\bar{q}$ **clusters**.
3. Have each cluster decay isotropically into pairs of hadrons, with rates determined by available phase space and density of states.



HERWIG had a clean theoretical justification (preconfinement), but in practice it has been less successful than PYTHIA at describing LHC data, particularly for baryon and heavy-quark production. The project has largely been abandoned, and is now only a topic of historical interest.

5 NLO Generators

PYTHIA is accurate at LO, but for precision predictions you need NLO or better. Several generators specialize in this:

- **SHERPA** – similar in spirit to PYTHIA (string-based hadronization), but does matrix elements at NLO out of the box.
- **MC@NLO** – pioneered the matching of NLO matrix elements with parton showers.
- **POWHEG** – another popular NLO-matching scheme, often combined with PYTHIA for the shower and hadronization.
- **DiPhoX** – specialized for photon pair production.

The Matching Problem

Why is NLO hard? The hard matrix element at NLO has real emission diagrams: an extra parton in the final state. But the parton shower *also* puts extra partons in the final state. If you're not careful, you double-count soft/collinear emissions. The schemes MC@NLO and POWHEG are different ways to fix this double counting. The details are mostly a computational issue and not essential to understand for our purposes – just be aware that “NLO + parton shower” is harder than it might sound.

6 HIJING: Heavy Ions

For heavy-ion collisions (pA , AA – nuclei hitting nuclei or protons hitting nuclei), you need a generator that understands nuclei. **HIJING** (“Heavy Ion Jet INteraction Generator”) is the traditional choice:

- It works by **superimposing** multiple nucleon-nucleon collisions generated by PYTHIA. Each NN collision is a PYTHIA event; HIJING combines many of them to simulate AA .
- It includes simple nuclear effects: initial-state nuclear shadowing of PDFs, jet quenching (energy loss in the medium), nuclear geometry.
- Importantly: standard HIJING does **not** include quark-gluon plasma (QGP) formation – it treats each collision as an independent event. Newer versions can include QGP effects.

Relevance to Heavy-Ion Physics

If you end up doing heavy-ion physics, HIJING is a standard tool but has limitations. Modern heavy-ion generators (e.g. AMPT, JEWEL, Q-PYTHIA) add more detailed treatments of the quark-gluon plasma and jet-medium interactions that HIJING lacks.

7 Summary: Who Does What

Generator	ME Order	Hadronization	Main Use Case
PYTHIA	LO	Lund strings	Default for pp at LHC
HERWIG	LO	Clusters	Retired; historical
SHERPA	NLO	Strings	High-multiplicity NLO
MC@NLO	NLO	depends	NLO matching pioneer
POWHEG	NLO	external	NLO+PS for specific processes
HIJING	LO+eikonal	PYTHIA-based	Heavy-ion collisions
DiPhoX	NLO	–	Photon pair production

8 Tying It All Back: The Generator's Output

What does a generator actually hand you? For each event, you get a list of particles in a standard format called an **event record**. Each particle has:

- A **PDG ID** code (standardized: 11 is e^- , 211 is π^+ , 2212 is p , -211 is π^- , etc.).
- A **4-momentum** (E, p_x, p_y, p_z).

- A **status flag** (is this particle final-state? intermediate? decayed?).
- Pointers to its **parents** and **daughters** in the event tree.

You then run this event record through a detector simulation (GEANT4, typically), which produces simulated detector hits – exactly the same format you’d get from real data. At that point, you can run your jet algorithm, your analysis code, your machine-learning classifiers, whatever. The generator + detector simulation gives you a complete, end-to-end dataset.

Why This Is Indispensable

Without event generators, you couldn’t:

- Train machine-learning classifiers (you need labeled training data with known truth).
- Calibrate jet energy scales (you need to know what the “true” jet energy is before detector smearing).
- Estimate backgrounds in BSM searches (you need to simulate SM processes you expect to dominate).
- Understand systematic uncertainties (you need to vary model parameters and see how predictions change).

Every experimental paper you read from ATLAS, CMS, LHCb, or ALICE depends on event generators. They are one of the quiet infrastructural triumphs of modern particle physics.

9 Summary of Part 3

Main Takeaways

1. An **event generator** is a Monte Carlo program that simulates particle collisions event by event, producing a list of stable final-state particles as output.
2. Every modern generator follows a four-step recipe: **hard process** → **parton shower** → **hadronization** → **hadron decays**. The first two are perturbative; the third is non-perturbative.
3. **PYTHIA** is the dominant LHC generator. It uses LO matrix elements, a p_T -ordered shower, and the Lund string model. Its parameters are tuned to data.
4. **HERWIG** uses the cluster model instead of strings; it is now largely retired.
5. **SHERPA**, **MC@NLO**, **POWHEG** are NLO generators, useful for precision predictions.
6. **HIJING** handles heavy-ion collisions by superimposing PYTHIA-generated NN collisions with nuclear corrections.
7. Event generators are the indispensable bridge between QCD theory and LHC data. Every experimental analysis relies on them.